

AI2301: Object Oriented Programming

List of Practical's:

1. Write a C++ program to implement a **class named Book** with the following data members:
Private: bookId, bookName, author, price
Public: getDetails(), printDetails()
The program should accept details of multiple books from the user, display the book information in **tabular form**, and calculate and display the **total price of all the books**.
2. Write a C++ program to implement a **class Book** with suitable data members. Use constructors to initialize book details and a destructor to indicate object destruction, thereby demonstrating the concepts of **objects, classes, constructors, and destructors**.
3. Design and implement a C++ program to model a banking system where the class BankAccount keeps track of the **total number of accounts created** and the **total balance across all accounts**. Use **static data members** to maintain these values and **static member functions** to retrieve and display them. Additionally, implement a member function in the BankAccount class that takes **another BankAccount object** as an argument and displays the **two account balances in ascending order**.
4. Write a C++ program to create a **Student class** with the following:
Private data members: Roll Number, Name, Marks of five subjects, Percentage, Result Division
Public members: Parameterized constructor, destructor, functions to calculate **percentage**, determine **grade/division**, and display student details

Implement a friend function `comparePercentage()` that takes **two student objects** and displays the one with the **higher percentage**.
5. Design a C++ **class Complex** with data members to store the real and imaginary parts. Provide **default and parameterized constructors**. Implement **operator overloading** for +, -, and * to perform arithmetic operations on complex numbers. Write a program that creates **two complex objects** and displays the results of these operations.
6. Identify the **commonalities and differences** between the classes **Publication, Book, and Magazine**. The classes share the following **common instance variables**: title, price, and copies, and a **common method**: saleCopy().
The **Book** class has an additional data member author and a method orderCopies().
The **Magazine** class has additional methods orderQty(), currentIssue(), and receiveIssue().
Write a program to:
 - a) Create objects of the classes.
 - b) Find and display the **number of copies ordered** for each book.
 - c) Calculate and display the **total sales of all publications**.
7. Design and implement a C++ **program** to demonstrate **inheritance** using the following employee hierarchy:
Base Class: Employee with data members: Emp_name, Emp_id, Address, Mail_id, and

Mobile_no.

Derived Classes: Programmer, TeamLead, AssistantProjectManager, and ProjectManager. Each derived class should have an additional data member **Basic Pay (BP)**.

Salary components for all employees are calculated as:

Dearness Allowance (DA): 52% of BP

House Rent Allowance (HRA): 27% of BP

Provident Fund (PF): 12% of BP

Staff Club Fund: 0.1% of BP

Implement member functions to **calculate gross and net salary** and **generate pay slips** for all employees.

8. Design a **base class Shape** with two **double-type data members** to represent the dimensions of the shape. Include member functions to **input the dimensions** and a **pure virtual function compute_area()** for calculating the area.

Derive two classes: **Triangle** and **Rectangle** from Shape and **override the compute_area()** function to calculate the area according to the specific shape.

Write a program that:

- Accepts the dimensions of a triangle or rectangle from the user.
 - Uses **dynamic binding** to call the appropriate compute_area() function.
 - Displays the calculated area of the shape.
9. Design and implement an **interface for Vehicles** to represent common functionalities. Consider vehicles such as **Bicycle, Bike, and Car**, which share the following functionalities:
- gearChange()
 - speedUp()
 - applyBrakes()

Create an **interface Vehicle** that declares these methods. Implement the interface in the classes **Bicycle, Bike, and Car**, providing **custom implementations** for each method according to the specific vehicle type.

10. Write a program that handles runtime errors such as **division by zero** and **invalid array index access** using built-in exception handling mechanisms. In addition, define and implement **custom exception classes** to manage application-specific errors such as **invalid input values** or **operation limits**, ensuring clear error messages and safe program execution.
11. Write a program to create a **Book class** and extend it by using **dynamic memory allocation**. Use the **new keyword** and **object pointers** to create **Book objects dynamically**. The program should allow the user to:
- Input details of one or more books.
 - Display the book details.
 - Demonstrate the use of **dynamic memory allocation** for objects.

12. Implement a **generic C++ program** using a C++ **Standard Library collection class**

(such as vector or list) to **count the number of elements** in the collection that satisfy a specific property. Example properties include:

Even numbers

Odd numbers

Prime numbers

Palindromes

The program should:

- a) Allow the user to **input the elements** of the collection.
- b) Allow the user to **choose a property** to count.
- c) **Display the total count** of elements satisfying the selected property.

13. Write a **C++ program** that implements a **generic function template** to **sort an array of any data type** using either **Bubble Sort** or **Selection Sort**.

The program should:

- a) Accept an array of any type (such as int, float, char, etc.) from the user.
- b) Use the **generic function template** to sort the array.
- c) Display the **sorted array**.

14. Design and implement **one AI-based Intelligent System** using **Object-Oriented Programming principles** for a real-world application of your choice. The system may address domains such as, **but not restricted to**, smart recommendation, intelligent grading and feedback, health monitoring, or resource management. The chosen application should analyze input data and apply rule-based or heuristic AI logic to produce meaningful predictions, classifications, or recommendations. The solution must be modular and extensible, clearly demonstrating effective use of abstraction, encapsulation, inheritance, and polymorphism.